Department of Electrical Engineering
University of Arkansas

ELEG 5173L Digital Signal Processing
Introduction to TMS320C6713 DSK

Dr. Jingxian Wu
wuj@uark.edu

ANALOG V.S DIGITAL

- **Analog signal processing (ASP)**
  - Analog signal processing is achieved by using analog components such as:
    - Resistor, capacitor, inductor
  - Sensitive to
    - inherent tolerances, temperature, voltage changes and mechanical vibrations
    - They can dramatically affect the effectiveness of the analogue circuitry
- **Digital signal processing (DSP)**
  - High precision
  - High tolerance to noise
  - Short development time
  - High flexibility
  - Low power consumption

Digital signal processing techniques are now so powerful that sometimes it is extremely difficult, if not impossible, for analogue signal processing to achieve similar performance.

DSP

- **Digital Signal Processor (DSP)**
  - A digital signal processor is a specialized microprocessor with an architecture optimized for the fast operational needs of digital signal processing
  - It is a special type of microprocessor
    - Have instructions designed specifically for digital signal processing
- **Use a DSP when the following are required:**
  - Cost saving.
  - Smaller size.
  - Low power consumption.
  - Processing of many “high” frequency signals in real-time.
- **Use a general purpose microprocessor (e.g. Intel Core 2 Duo) when the following are required:**
  - Large memory.
  - Advanced operating systems.

DSP

- **DSP**
  - The key operation in digital signal processing is sum of products (SOP) or multiply and accumulation (MAC)

Algorithm	Equation
Finite Impulse Response Filter	$y(n)=\sum_{k=0}^{M} a_k\, x(n-k)$
Infinite Impulse Response Filter	$y(n)=\sum_{k=0}^{M} a_k\, x(n-k)+\sum_{k=1}^{N} b_k\, y(n-k)$
Convolution	$y(n)=\sum_{k=0}^{N} x(k)h(n-k)$
Discrete Fourier Transform	$X(k)=\sum_{n=0}^{N-1} x(n)\exp[-j(2\pi/N)nk]$
Discrete Cosine Transform	$F(u)=\sum_{x=0}^{N-1} c(u).f(x).\cos\left[\frac{\pi}{2N}u(2x+1)\right]$

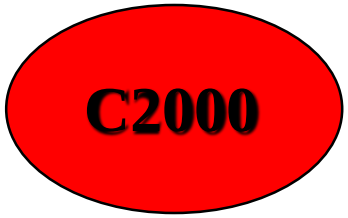
DSP

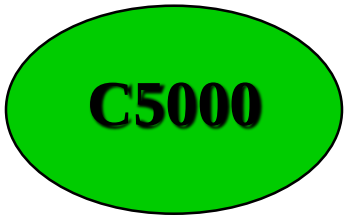
- **DSP are optimized for MAC operations**
  - Multiplication and addition are done in hardware and in one cycle.
  - Example: 4-bit multiply (unsigned).

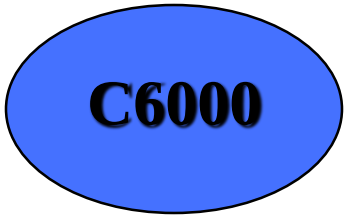
Hardware	Microcode	
1011	1011	
x 1110	x 1110	
10011010	0000	Cycle 1
	1011.	Cycle 2
	1011..	Cycle 3
	1011...	Cycle 4
	10011010	Cycle 5

DSP general purpose microprocessor

TEXAS INSTRUMENTS' TMS320 FAMILY







Lowest Cost

Control Systems

- Motor Control
- Storage
- Digital Ctrl Systems

Efficiency

Best MIPS per Watt / Dollar / Size

- Wireless phones
- Internet audio players
- Digital still cameras
- Modems
- Telephony
- VoIP

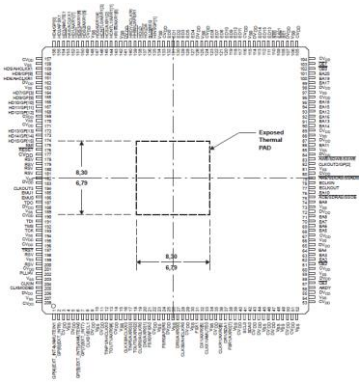
Performance & Best Ease-of-Use

- **Multi Channel and Multi Function App's**
- Comm Infrastructure
- Wireless Base-stations
- DSL
- Imaging
- Multi-media Servers
- Video

TMS320C6713

- **TMS320C6713**
  - Very-long-instruction-word (VLIW) architecture
    - Suited for numerically intensive algorithms.
  - 8 instructions can be fetched per clock cycle
    - E.g. clock rate 225 MHz, clock period = 1/225 MHz = 4.44 ns
    - 8 instructions can be fetched every 4.44 ns.
  - 264 KB internal memory.
  - 6 Arithmetic and logic units (ALUs)
  - 2 multiplier units
  - 32 bit address bus (up to 4 GB memory)
  - 2 sets of 32-bit registers
  - Capable of both fixed point and floating point processing.



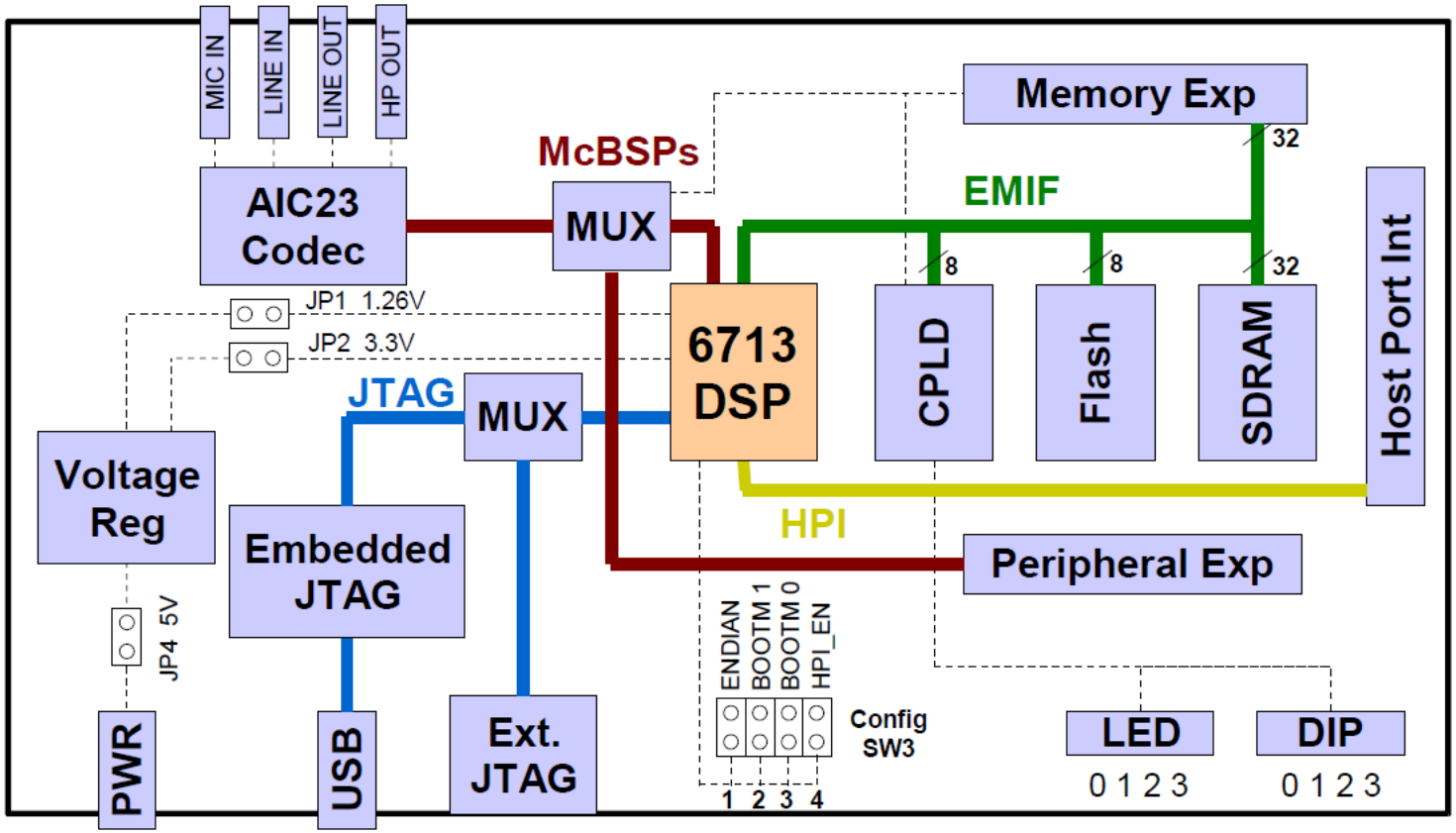


TMS320C6713 DSK

- **TMS320C6713 DSK**
  - a low-cost development platform designed to speed the development of applications based on TI´s TMS320C6713 DSP
  - 512K Flash and 16MB SDRAM
  - DSP operating at 225 MHz
  - An AIC32 stereo codec

CPLD: Complex Programmable Logic Device

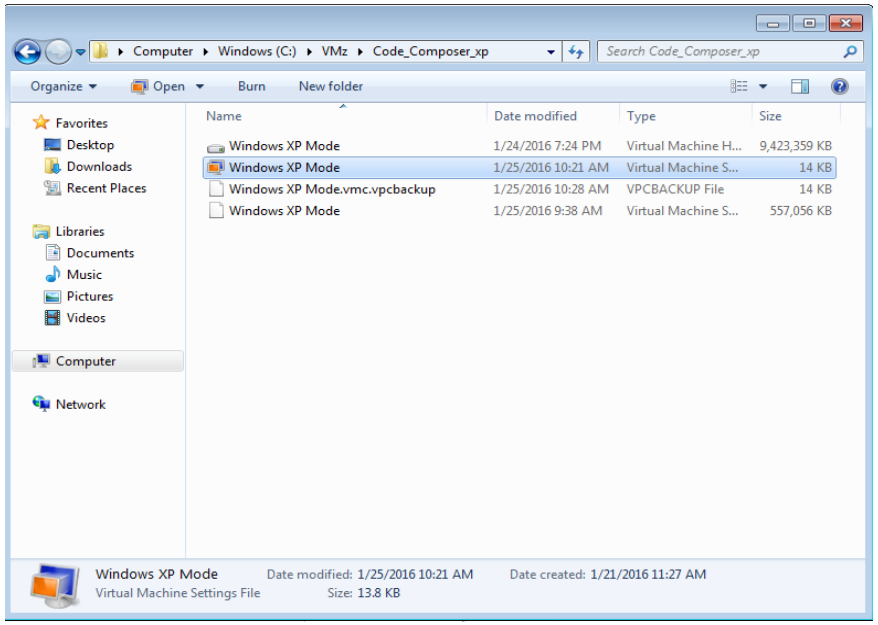
McBSP: Multi-channel buffered serial ports



DEVELOPMENT ENVIROMENT

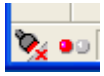
- **Getting started:**
  - 1. launch windows XP mode

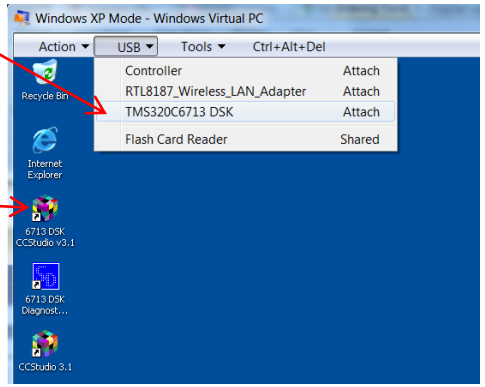
    Start → Computer→ windows (C:)→ VMz → Code_Composer_xp → windows XP Mode



  - 2. plug the DSK to PC through USB port, power DSK on.

DEVELOPMENT ENVIROMENT (cont'd)

- **Getting started:**
  - 3. in Windows XP mode, USB → TMS320C6713DSK Attach
  - 4. on Windows XP mode desktop, launch 6713DSK CCStudio V3.1
  - 5. If a "Waiting for USB enumeration" dialog box show up, repeat step 3.
  - 6. In CC Studio 3.1, Debug → Connect. If successful, the left lower corner of CC Studio 3.1 should be changed from  to 



  - If any of the above step does not work, try to unplug DSK, shut down Windows XP mode, and try steps 1 – 6 again.

EXAMPLE

- **sin8_LED.c**

```
//sine8_LED.c  sine generation with DIP switch control

#include "dsk6713_aic23.h"                  //codec support
Uint32 fs = DSK6713_AIC23_FREQ_8KHZ;        //set sampling rate
#define DSK6713_AIC23_INPUT_MIC 0x0015
#define DSK6713_AIC23_INPUT_LINE 0x0011
Uint16 inputsource=DSK6713_AIC23_INPUT_MIC;//select input
#define LOOPLENGTH 8
short loopindex = 0;                        //table index
short gain = 10;                            //gain factor
short sine_table[LOOPLENGTH]={0,707,1000,707,0,-707,-1000,-707}; //sine values

void main()
{
  comm_poll();                              //init DSK,codec,McBSP
  DSK6713_LED_init();                       //init LED from BSL
  DSK6713_DIP_init();                       //init DIP from BSL
  while(1)                                  //infinite loop
  {
    if(DSK6713_DIP_get(0)==0)               //=0 if DIP switch #0 pressed
    {
      DSK6713_LED_on();                     //turn LED #0 ON
      output_left_sample(sine_table[loopindex++]*gain); //output sample
      if (loopindex >= LOOPLENGTH) loopindex = 0; //reset table index
    }
    else DSK6713_LED_off(0);                //turn LED off if not pressed
  }                                         //end of while(1) infinite loop
}                                           //end of main
```

EXAMPLE

- **sin8_buf.c**
  - AIC23 will generate an interrupt at every sample instant
  - fs = 8KHz: 8,000 interrupts will be generated by the AIC23
  - Every time an interrupt is generated, c_int11 will be executed.

```
//sine8_buf.c sine generation with output stored in buffer
#include "DSK6713_AIC23.h"                    //codec support
Uint32 fs=DSK6713_AIC23_FREQ_8KHZ;            //set sampling rate
#define DSK6713_AIC23_INPUT_MIC 0x0015
#define DSK6713_AIC23_INPUT_LINE 0x0011
Uint16 inputsource=DSK6713_AIC23_INPUT_MIC; // select input
#define LOOPLENGTH 8
#define BUFFERLENGTH 256
int loopindex = 0;                            //table index
int bufindex = 0;                             //buffer index
short sine_table[LOOPLENGTH]={0,707,1000,707,0,-707,-1000,-707};
int out_buffer[BUFFERLENGTH];                 //output buffer
short gain = 10;

interrupt void c_int11()                      //interrupt service routine
{
  short out_sample;

  out_sample = sine_table[loopindex++]*gain;
  output_left_sample(out_sample);             //output sample value
  out_buffer[bufindex++] = out_sample;        //store in buffer
  if (loopindex >= LOOPLENGTH) loopindex = 0; //check for end of table
  if (bufindex >= BUFFERLENGTH) bufindex = 0; //check for end of buffer
  return;                                     //return from interrupt
}

void main()
{
  comm_intr();  //initialise DSK
  while(1);     //infinite loop
}
```